

Formal Semantics of `md2arxiv`

Denotational and Axiomatic

Fabio F.G. Buono

2026

Contents

1	Introduction	2
2	Abstract Syntax	2
2.1	The source language	2
2.2	The target language	3
3	Denotational Semantics: Base Level	3
3.1	Domains and meaning functions	3
3.2	The escape function	3
3.3	Semantics of inlines	4
3.4	Semantics of blocks	5
3.5	Semantics of the document body	7
4	Denotational Semantics: CPO Domains and Fixed Points	7
4.1	Motivation	7
4.2	Complete partial orders	7
4.3	The CPO of strings with prefix order	8
4.4	Concatenation as a continuous function	8
4.5	The function $\mathcal{E}[\![\cdot]\!]$ as a least fixed point	9
5	Axiomatic Semantics	9
5.1	The Hoare system	9
5.2	The function <code>read_until</code>	10
5.3	The function <code>try_delim</code>	11
5.4	Global invariant of the lexer	12
6	Conclusions	13
	Appendices	14
A	Free Monoids and Formal Languages	14
B	Least Fixed Point and Kleene's Theorem	14

1 Introduction

`md2arxiv` is a transpiler from academic Markdown to $\text{\LaTeX}$ organized as a miniature compilation pipeline in which a lexer analyses inline elements, a parser builds the block structure, an emission module translates the syntax tree into $\text{\LaTeX}$ text, and a validator checks the consistency of bibliographic citations. Compact as the project is, each phase has a precise semantics that lends itself to a formal treatment, and this document aims to provide it according to two classical approaches.

Denotational semantics assigns a mathematical meaning to every construct of the source language by induction on the structure of the syntax tree and is natural for the pure emission functions; for the style I tried to follow that of the lecture notes of the languages course at the University of Pisa. Axiomatic semantics instead describes the behaviour of stateful functions through preconditions and postconditions linked by derivation rules, and this is well suited to the imperative functions of the lexer that modify a cursor and a buffer. We note that syntactic domains are introduced through BNF grammars, semantic domains are defined as complete partial orders, and meaning functions are given by clauses on the syntactic structure.

2 Abstract Syntax

2.1 The source language

To assign a meaning to a program it is necessary to describe its syntactic structure precisely, setting aside the concrete details of the textual representation and highlighting only the structural relationships among the constructs. This description is called *abstract syntax* and constitutes the starting point of any formal semantic treatment. The source language accepted by `md2arxiv` is a subset of Markdown and its abstract syntax is organised on two hierarchical levels that reflect the structure of the `ast.ml` module: the *inline* elements, which live within a line of text and express its local formatting, and the *block* elements, which constitute the backbone of the document at the level of paragraphs, sections, and environments.

Definition 2.1 (Abstract syntax of inlines). The syntactic domain **Inline** is defined by the following BNF grammar, where s , k , t , and u denote elements of **String**.

i	$::=$	<code>Text(s)</code>	raw text
		<code>Bold(s)</code>	bold
		<code>Italic(s)</code>	italic
		<code>Code(s)</code>	inline code
		<code>Cite(k)</code>	bibliographic citation
		<code>Link(t, u)</code>	hyperlink

An *inline sequence* is an element of **Inline**^{*}, the set of finite lists of elements of **Inline**.

The **Block** domain describes the structural constructs of the document. Among these, **Heading** carries the hierarchical level as a natural number, **UnorderedList** is a list of inline sequences, **Table** explicitly separates the header from the data rows, and **Bibliography** is a list of key-description pairs that corresponds directly to the `thebibliography` environment of $\text{\LaTeX}$.

Definition 2.2 (Abstract syntax of blocks). The syntactic domain **Block** is defined by the

following BNF grammar.

$b ::=$	$\text{Heading}(n, \mathbf{i})$	$n \in \mathbb{N}, n \geq 1, \mathbf{i} \in \mathbf{Inline}^*$
	$\text{Paragraph}(\mathbf{i})$	$\mathbf{i} \in \mathbf{Inline}^*$
	$\text{UnorderedList}(\mathbf{i}_1, \dots, \mathbf{i}_k)$	$\mathbf{i}_j \in \mathbf{Inline}^*, k \geq 0$
	$\text{CodeBlock}(s)$	$s \in \mathbf{String}$
	$\text{Image}(a, p)$	$a, p \in \mathbf{String}$
	$\text{Table}(H, R)$	$H \in \mathbf{String}^+, R \in (\mathbf{String}^*)^*$
	$\text{Bibliography}((k_1, d_1), \dots, (k_m, d_m))$	$k_j, d_j \in \mathbf{String}$

A *document* Doc is a pair $(\mu, \mathbf{b})$ where μ is a metadata record and $\mathbf{b} \in \mathbf{Block}^*$ is the list of blocks forming the body.

Remark 2.1. The constructor $\text{Heading}(n, \mathbf{i})$ places no upper bound on n , and the emission function maps every $n \geq 3$ to `\subsubsection`. The requirement $H \in \mathbf{String}^+$ for tables guarantees that every table has at least one column in the header, while the row set R may be empty, producing a header-only table.

2.2 The target language

The target language of `md2arxiv` is **String**, the set of all finite strings over the Unicode alphabet, with no syntactic structure of its own. The $\text{\LaTeX}$ fragments produced by the emission functions are strings that acquire meaning only when interpreted by the typesetting engine, and the denotational semantics we construct is a family of functions from structured syntactic domains to this flat domain, with compositionality as the central property.

3 Denotational Semantics: Base Level

3.1 Domains and meaning functions

The denotational approach consists in associating to each syntactic phrase a mathematical object, called its *denotation*, that captures its meaning independently of its internal structure. In the case of `md2arxiv`, the denotation of each construct is a $\text{\LaTeX}$ string, and the meaning of an inline is the text fragment it represents, the meaning of a block is the corresponding $\text{\LaTeX}$ environment, and the meaning of a list of blocks is their concatenation. The translation is *compositional* in the sense that the meaning of a term depends solely on the meaning of its immediate subterms, and the emission function has neither memory nor global state.

The main semantic domain is **String** with concatenation $\cdot$, which forms a monoid $(\mathbf{String}, \cdot, \varepsilon)$ with neutral element the empty string ε . The meaning functions that constitute the semantics of `md2arxiv` are the following.

Definition 3.1 (Meaning functions). The denotational semantics of `md2arxiv` is given by the family of functions

$$\begin{aligned} \mathcal{S} : \mathbf{String} &\rightarrow \mathbf{String}, & \mathcal{I}[\![\cdot]\!] : \mathbf{Inline} &\rightarrow \mathbf{String}, & \mathcal{I}^*[\![\cdot]\!] : \mathbf{Inline}^* &\rightarrow \mathbf{String}, \\ \mathcal{B}[\![\cdot]\!] : \mathbf{Block} &\rightarrow \mathbf{String}, & \mathcal{E}[\![\cdot]\!] : \mathbf{Block}^* &\rightarrow \mathbf{String}. \end{aligned}$$

3.2 The escape function

Raw text appearing in a Markdown document may contain characters that carry special meaning in $\text{\LaTeX}$, such as `\&`, `\%`, `\$`, and others. Before inserting an arbitrary string into a

$\text{\LaTeX}$ document one must therefore protect it by replacing each special character with the corresponding escape sequence. This operation is performed by the function $\mathcal{S}$, defined character by character as a monoid homomorphism on strings.

Definition 3.2 (Escape function). The function $\mathcal{S} : \mathbf{String} \rightarrow \mathbf{String}$ is defined by extending as a homomorphism the following function on individual characters $c \in \Sigma$.

$$\mathcal{S}(c) = \begin{cases} \backslash\& & c = \& \\ \backslash\% & c = \% \\ \backslash\$ & c = \$ \\ \backslash\# & c = \# \\ \backslash_ & c = _ \\ \backslash\{ & c = \{ \\ \backslash\} & c = \} \\ \backslash\text{asciitilde}\{ & c = \sim \\ \backslash\text{asciicircum}\{ & c = \^{} \\ \backslash\text{backslash}\{ & c = \backslash \\ c & \text{otherwise} \end{cases}$$

For a string $s = c_1 c_2 \cdots c_n$ one has $\mathcal{S}(s) = \mathcal{S}(c_1) \cdot \mathcal{S}(c_2) \cdots \mathcal{S}(c_n)$, with $\mathcal{S}(\varepsilon) = \varepsilon$.

Proposition 3.1. $\mathcal{S}$ is a homomorphism of the monoid $(\mathbf{String}, \cdot, \varepsilon)$ into itself, that is, $\mathcal{S}(s \cdot t) = \mathcal{S}(s) \cdot \mathcal{S}(t)$ for every $s, t \in \mathbf{String}$.

Proof. The character-by-character definition and the extension by concatenation make the property immediate by induction on the length of s . $\square$

3.3 Semantics of inlines

The core of the denotational semantics is the function $\mathcal{I}[\![\cdot]\!]$, which assigns to each constructor of **Inline** a $\text{\LaTeX}$ fragment through independent cases, reflecting compositionality at the level of algebraic data types.

Definition 3.3 (Meaning function for inlines). The function $\mathcal{I}[\![\cdot]\!] : \mathbf{Inline} \rightarrow \mathbf{String}$ is defined by cases on the constructor.

$$\begin{aligned} \mathcal{I}[\![\text{Text}(s)]\!] &= \mathcal{S}(s) \\ \mathcal{I}[\![\text{Bold}(s)]\!] &= \backslash\text{textbf}\{ \cdot \mathcal{S}(s) \cdot \} \\ \mathcal{I}[\![\text{Italic}(s)]\!] &= \backslash\text{emph}\{ \cdot \mathcal{S}(s) \cdot \} \\ \mathcal{I}[\![\text{Code}(s)]\!] &= \backslash\text{texttt}\{ \cdot \mathcal{S}(s) \cdot \} \\ \mathcal{I}[\![\text{Cite}(k)]\!] &= \backslash\text{cite}\{ \cdot k \cdot \} \\ \mathcal{I}[\![\text{Link}(t, u)]\!] &= \backslash\text{href}\{ \cdot u \cdot \} \{ \cdot \mathcal{S}(t) \cdot \} \end{aligned}$$

Remark 3.1. We note two asymmetries in the definition. In $\text{Cite}(k)$ the key k is not passed through $\mathcal{S}$, because bibliographic keys are synthetic $\text{\LaTeX}$ identifiers that contain no special characters by construction. In $\text{Link}(t, u)$ the text t is escaped but the URL u is not, because the `hyperref` package handles special characters in URLs internally and a further escape would produce malformed output.

The function $\mathcal{I}^*[\![\cdot]\!]$ extends $\mathcal{I}[\![\cdot]\!]$ to inline sequences by simple concatenation of the meanings of individual elements, which is the most elementary form of compositionality.

Definition 3.4 (Semantics of an inline sequence). The function $\mathcal{I}^*[\![\cdot]\!] : \mathbf{Inline}^* \rightarrow \mathbf{String}$ is defined by structural induction on the list.

$$\mathcal{I}^*[\![\]\!] = \varepsilon \quad \text{and} \quad \mathcal{I}^*[\![i :: i]\!] = \mathcal{I}[\![i]\!] \cdot \mathcal{I}^*[\![i]\!].$$

Example 3.1. Consider the sequence representing the phrase “see [hoare1969].” in the source document, that is $[\text{Text}(\text{“see ”}), \text{Cite}(\text{“hoare1969”}), \text{Text}(\text{“.”})]$. Applying the definition one obtains:

$$\begin{aligned} \mathcal{I}^*[\![\text{Text}(\text{“see ”}), \text{Cite}(\text{“hoare1969”}), \text{Text}(\text{“.”})]\!] \\ &= \mathcal{S}(\text{“see ”}) \cdot \backslash\text{cite}\{\text{hoare1969}\} \cdot \mathcal{S}(\text{“.”}) \\ &= \text{“see \text{cite}\{\text{hoare1969}\}.”} \end{aligned}$$

3.4 Semantics of blocks

The function $\mathcal{B}[\![\cdot]\!]$ constitutes the richest level of the base denotational semantics. Each constructor of **Block** corresponds to a L^AT_EX environment or command, and the case-by-case definition faithfully mirrors the structure of the OCaml code in `emit.ml`. For **Heading** and **Paragraph** the subterms are inline sequences whose semantics is already given by $\mathcal{I}^*[\![\cdot]\!]$; for **UnorderedList** they are a list of sequences concatenated after prepending the `\item` marker; and for **Table** the definition relies on an auxiliary normalisation function that compensates for any column-count inconsistencies reported by the parser.

Definition 3.5 (Meaning function for blocks). The function $\mathcal{B}[\![\cdot]\!] : \mathbf{Block} \rightarrow \mathbf{String}$ is defined by cases on the constructor, with each case relying on $\mathcal{I}^*[\![\cdot]\!]$ for the treatment of inline subterms.

Heading. The hierarchical level determines the L^AT_EX command through the auxiliary function $cmd : \mathbb{N} \rightarrow \mathbf{String}$ defined by $cmd(1) = \backslash\text{section}$, $cmd(2) = \backslash\text{subsection}$, and $cmd(n) = \backslash\text{subsubsection}$ for $n \geq 3$. Then:

$$\mathcal{B}[\![\text{Heading}(n, i)]\!] = cmd(n) \cdot \{ \cdot \mathcal{I}^*[\![i]\!] \cdot \} \cdot \backslash n \backslash n$$

Paragraph.

$$\mathcal{B}[\![\text{Paragraph}(i)]\!] = \mathcal{I}^*[\![i]\!] \cdot \backslash n \backslash n$$

UnorderedList. We denote by $\bigodot_{j=1}^k$ the ordered concatenation of the elements of an indexed sequence. Then:

$$\begin{aligned} \mathcal{B}[\![\text{UnorderedList}(i_1, \dots, i_k)]\!] &= \backslash\text{begin}\{\text{itemize}\} \backslash n \\ &\quad \cdot \bigodot_{j=1}^k (\backslash\text{item} \cdot \mathcal{I}^*[\![i_j]\!] \cdot \backslash n) \\ &\quad \cdot \backslash\text{end}\{\text{itemize}\} \backslash n \backslash n \end{aligned}$$

CodeBlock. The content s is not passed through $\mathcal{S}$ because the `verbatim` environment of $\text{\LaTeX}$ treats its content as raw text, without interpreting escape sequences.

$$\mathcal{B}[\text{CodeBlock}(s)] = \text{\texttt{\backslashbegin\{verbatim\}\n\cdot s\cdot\n\end\{verbatim\}\n\n}}$$

Image.

$$\begin{aligned} \mathcal{B}[\text{Image}(a, p)] = & \text{\texttt{\backslashbegin\{figure\}[h]\n}} \\ & \cdot \text{\texttt{\backslashcentering\n}} \\ & \cdot \text{\texttt{\backslashincludegraphics[width = 0.8\linewidth]\{ \cdot p \cdot \}\n}} \\ & \cdot \text{\texttt{\backslashcaption\{ \cdot \mathcal{S}(a) \cdot \}\n}} \\ & \cdot \text{\texttt{\backslashend\{figure\}\n\n}} \end{aligned}$$

Table. Let $c = |H|$ be the number of columns in the header. Since the parser may produce rows with a number of cells different from c and emit a warning, the emission function normalises each row through $\nu_c : \text{String}^* \rightarrow \text{String}^c$, which brings each row to c cells by truncating longer ones and padding shorter ones with ε .

$$\nu_c(r) = \begin{cases} (r_1, \dots, r_c) & |r| \geq c \\ (r_1, \dots, r_{|r|}, \underbrace{\varepsilon, \dots, \varepsilon}_{c-|r|}) & |r| < c \end{cases}$$

The row function, with $H = (h_1, \dots, h_c)$ and $R = (r_1, \dots, r_m)$, is defined by

$$\text{row}(s_1, \dots, s_c) = \mathcal{S}(s_1) \cdot \& \cdot \dots \cdot \& \cdot \mathcal{S}(s_c) \cdot \text{\texttt{\backslash\backslash\n}}.$$

Then:

$$\begin{aligned} \mathcal{B}[\text{Table}(H, R)] = & \text{\texttt{\backslashbegin\{center\}\n}} \\ & \cdot \text{\texttt{\backslashbegin\{tabular\}\{ \cdot \underbrace{1 \dots 1}_c \cdot \}\n}} \\ & \cdot \text{\texttt{\backslashtoprule\n\cdot row(H) \cdot \backslashmidrule\n}} \\ & \cdot \bigcirc_{j=1}^m \text{row}(\nu_c(r_j)) \\ & \cdot \text{\texttt{\backslashbottomrule\n\cdot \backslashend\{tabular\}\n\backslashend\{center\}\n\n}} \end{aligned}$$

Bibliography.

$$\begin{aligned} \mathcal{B}[\text{Bibliography}((k_1, d_1), \dots, (k_m, d_m))] = & \text{\texttt{\backslashbegin\{thebibliography\}\{99\}\n}} \\ & \cdot \bigcirc_{j=1}^m (\text{\texttt{\backslashbibitem\{ \cdot k_j \cdot \} \cdot \mathcal{S}(d_j) \cdot \}\n}) \\ & \cdot \text{\texttt{\backslashend\{thebibliography\}\n\n}} \end{aligned}$$

3.5 Semantics of the document body

The function $\mathcal{E}[\cdot]$ completes the base semantics by handling the list of blocks that constitutes the document body. Its meaning is the concatenation of the meanings of the individual blocks in the order they appear, and its definition by structural induction is the most direct possible.

Definition 3.6 (Meaning function for the body). The function $\mathcal{E}[\cdot] : \mathbf{Block}^* \rightarrow \mathbf{String}$ is defined by structural induction on the list.

$$\mathcal{E}[\cdot] = \varepsilon \quad \text{and} \quad \mathcal{E}[b :: \mathbf{b}] = \mathcal{B}[b] \cdot \mathcal{E}[\mathbf{b}].$$

Proposition 3.2 (Compositionality). *The functions $\mathcal{I}[\cdot]$, $\mathcal{B}[\cdot]$, and $\mathcal{E}[\cdot]$ are compositional in the sense that the meaning of each term is determined solely by its constructor and the meaning of its immediate subterms, without ever referring to the internal structure of the latter.*

Proof. The property is direct from the structure of the case-by-case definitions, since each clause invokes $\mathcal{I}[\cdot]$ or $\mathcal{B}[\cdot]$ exclusively on the immediate subterms of the current term, without ever accessing their internal structure. $\square$

4 Denotational Semantics: CPO Domains and Fixed Points

4.1 Motivation

The functions defined in the previous section are total, since both algebraic types and lists are finite structures and the emission functions traverse them by well-founded structural recursion. With this in mind we note that the theory of CPOs, introduced by Dana Scott and developed systematically by Winskel [3], provides the mathematical framework within which denotational semantics extends uniformly to partial functions, general recursion, and infinite types. In this section we place the emission functions in this more general setting, showing that $\mathcal{E}[\cdot]$ is the least fixed point of a continuous operator on an appropriate CPO of strings, and clarifying in what sense structural recursion on lists is a special case of the general fixed-point theory.

4.2 Complete partial orders

A complete partial order, or CPO, is the minimal mathematical structure on which recursion can be interpreted as the computation of a fixed point. The central idea is that the elements of a CPO represent *approximations* of a value, with the least element $\perp$ representing the total absence of information, and the order capturing the notion of “greater information”. A recursive function is then the limit of a chain of successive approximations, and continuity guarantees that this limit exists and is itself a fixed point.

Definition 4.1 (Partial order and CPO). A *partial order* is a pair $(D, \sqsubseteq)$ where $\sqsubseteq$ is a reflexive, transitive, and antisymmetric relation on D . A *CPO* is a partial order in which every chain $d_0 \sqsubseteq d_1 \sqsubseteq d_2 \sqsubseteq \dots$ has a least upper bound $\bigsqcup_{n \geq 0} d_n \in D$.

Definition 4.2 (Flat CPO). Given a set A , the *flat CPO* $A_\perp$ is obtained by adding to A a minimal element $\perp$ with order $a \sqsubseteq b$ if and only if $a = \perp$ or $a = b$. The element $\perp$ represents a result not yet computed, or undefined.

Definition 4.3 (Monotone and continuous functions). A function $f : D \rightarrow E$ between CPOs is called *monotone* if $d \sqsubseteq d'$ implies $f(d) \sqsubseteq f(d')$, and *continuous* if it is monotone and

preserves least upper bounds of chains, in the sense that $f(\bigsqcup_n d_n) = \bigsqcup_n f(d_n)$ for every chain (d_n) .

Definition 4.4 (Least fixed point). Let D be a CPO with least element $\perp$ and let $f : D \rightarrow D$ be continuous. The *least fixed point* of f is $\text{fix}(f) = \bigsqcup_{n \geq 0} f^n(\perp)$, where $f^0(\perp) = \perp$ and $f^{n+1}(\perp) = f(f^n(\perp))$.

Proposition 4.1 (Knaster-Tarski Theorem). *Under the conditions of the preceding definition, $\text{fix}(f)$ is the least fixed point of f , in the sense that it satisfies $f(\text{fix}(f)) = \text{fix}(f)$ and is the least element with this property.*

Proof. The fixed-point property follows from continuity: $f(\bigsqcup_n f^n(\perp)) = \bigsqcup_n f^{n+1}(\perp) = \bigsqcup_n f^n(\perp)$. If d is another fixed point, then $\perp \sqsubseteq d$ and by induction $f^n(\perp) \sqsubseteq f^n(d) = d$ for every n , from which $\text{fix}(f) = \bigsqcup_n f^n(\perp) \sqsubseteq d$. $\square$

4.3 The CPO of strings with prefix order

To apply the theory of CPOs to the emission functions one must equip **String** with a partial order structure. The most natural choice for strings is the prefix order, in which s is less than t when t is obtained from s by appending characters; this order captures the idea that a partially built string is an approximation of the complete string, and that adding characters amounts to gaining information.

Definition 4.5 (Prefix order on $\text{String}_\perp$). Let $\text{String}_\perp = \text{String} \cup \{\perp\}$. The order $\sqsubseteq$ on $\text{String}_\perp$ is defined by $s \sqsubseteq t$ if and only if $s = \perp$, or $s, t \in \text{String}$ and $\exists u \in \text{String}$ such that $t = s \cdot u$.

Proposition 4.2. $(\text{String}_\perp, \sqsubseteq)$ is a CPO with least element $\perp$.

Proof. Reflexivity follows from $s = s \cdot \varepsilon$. For transitivity, if $t = s \cdot u$ and $r = t \cdot v$ then $r = s \cdot (u \cdot v)$. For antisymmetry, if $t = s \cdot u$ and $s = t \cdot v$ then $s = s \cdot u \cdot v$, from which $u = v = \varepsilon$ and $s = t$. Finally, chains of finite strings are stationary, so every chain has a least upper bound. $\square$

4.4 Concatenation as a continuous function

To use $\cdot$ in the definition of the functional of $\mathcal{E}$ within the CPO $\text{String}_\perp$, it is convenient to extend it to handle the element $\perp$ and verify that the resulting function is continuous.

Definition 4.6 (Strict concatenation on $\text{String}_\perp$). The strict concatenation $\cdot_\perp : \text{String}_\perp \times \text{String}_\perp \rightarrow \text{String}_\perp$ is defined by

$$s \cdot_\perp t = \begin{cases} \perp & \text{if } s = \perp \text{ or } t = \perp \\ s \cdot t & \text{otherwise.} \end{cases}$$

Proposition 4.3. *The functions $\lambda s. s \cdot_\perp t$ and $\lambda t. s \cdot_\perp t$ are continuous on $(\text{String}_\perp, \sqsubseteq)$ for every fixed value of the other argument.*

Proof. Chains of defined strings in $\text{String}_\perp$ are stationary: if (s_n) is a chain and $s_N \in \text{String}$ for some N , then $s_n = s_N$ for all $n \geq N$. Continuity follows from monotonicity and the stationarity of chains. $\square$

4.5 The function $\mathcal{E}[\cdot]$ as a least fixed point

The functional $\Phi : [\mathbf{Block}^* \rightarrow \mathbf{String}_\perp] \rightarrow [\mathbf{Block}^* \rightarrow \mathbf{String}_\perp]$ transforms an approximated function into one that can handle lists with one more element, adding the ability to process the head of the list and delegating the rest to the argument function.

Definition 4.7 (Functional of $\mathcal{E}$). The functional Φ is defined by setting, for every $f \in [\mathbf{Block}^* \rightarrow \mathbf{String}_\perp]$ and every list $\mathbf{b}$,

$$\Phi(f)(\mathbf{b}) = \begin{cases} \varepsilon & \mathbf{b} = [] \\ \mathcal{B}_\perp[b] \cdot_\perp f(\mathbf{b}') & \mathbf{b} = b :: \mathbf{b}' \end{cases}$$

where $\mathcal{B}_\perp[\cdot]$ is the version of $\mathcal{B}[\cdot]$ that uses $\cdot_\perp$ in place of $\cdot$.

Proposition 4.4. Φ is a continuous operator on the pointwise-ordered space $[\mathbf{Block}^* \rightarrow \mathbf{String}_\perp]$.

Proof. Monotonicity follows from the monotonicity of $\cdot_\perp$: if $f \sqsubseteq g$ then $\mathcal{B}_\perp[b] \cdot_\perp f(\mathbf{b}') \sqsubseteq \mathcal{B}_\perp[b] \cdot_\perp g(\mathbf{b}')$. Continuity is a consequence of the continuity of $\cdot_\perp$ and the pointwise structure of the limit in the function space. $\square$

Proposition 4.5 ($\mathcal{E}[\cdot]$ as a least fixed point). $\mathcal{E}[\cdot]$ is the least fixed point of Φ , that is, $\mathcal{E}[\cdot] = \text{fix}(\Phi) = \bigsqcup_{n \geq 0} \Phi^n(\lambda \mathbf{b}. \perp)$.

Proof. Let $\perp_0 = \lambda \mathbf{b}. \perp$. The first approximations are:

$$\begin{aligned} \Phi^0(\perp_0)(\mathbf{b}) &= \perp \quad \text{for every } \mathbf{b} \\ \Phi^1(\perp_0)(\mathbf{b}) &= \begin{cases} \varepsilon & \mathbf{b} = [] \\ \perp & |\mathbf{b}| \geq 1 \end{cases} \\ \Phi^2(\perp_0)(\mathbf{b}) &= \begin{cases} \varepsilon & \mathbf{b} = [] \\ \mathcal{B}_\perp[b] & \mathbf{b} = [b] \\ \perp & |\mathbf{b}| \geq 2 \end{cases} \end{aligned}$$

By induction on n , $\Phi^n(\perp_0)(\mathbf{b}) = \mathcal{E}[\mathbf{b}]$ if $|\mathbf{b}| < n$ and $\perp$ if $|\mathbf{b}| \geq n$. Each $\mathcal{E}[\mathbf{b}]$ is reached in $|\mathbf{b}| + 1$ steps, and the limit of the increasing chain $(\Phi^n(\perp_0))_{n \geq 0}$ is the total function $\mathcal{E}[\cdot]$, which by the Knaster-Tarski Theorem is the least fixed point of Φ . $\square$

Remark 4.1. The fact that $\mathcal{E}[\cdot]$ is already total and that the fixed point collapses onto $\mathbf{Block}^* \rightarrow \mathbf{String} \subset \mathbf{String}_\perp$ is a consequence of the finiteness of lists and well-founded structural recursion. The CPO treatment adds conceptual clarity, clarifying the sense in which successive approximations converge to the total function and showing that structural recursion on lists is the limiting case of the general theory, in which the fixed point is reached in a finite number of steps equal to the length of the list plus one.

5 Axiomatic Semantics

5.1 The Hoare system

Axiomatic semantics describes the behaviour of a program as a relation between states before and after execution. The approach was introduced by Floyd [6] for flow diagrams

and formalised by Hoare [5] for structured programming languages in what is today the universally adopted form of the *Hoare triple*

$$\{P\} C \{Q\},$$

where P is the *precondition*, C the command, and Q the *postcondition*. The triple expresses *partial correctness*, in the sense that if the precondition P holds before the execution of C and C terminates, then the postcondition Q holds afterwards. Total correctness additionally requires proving that C always terminates when P is satisfied, but for the functions we consider termination is evident from the code structure.

We apply this schema to the stateful functions of the `lexer.ml` module. The relevant state consists of the cursor i , an integer reference indicating the current position in the input string s of length ℓ , and the accumulation buffer `buf`. We use the notation i_0 for the value of i at function entry, and $!i$ for its value at exit.

5.2 The function `read_until`

The function `read_until` scans the string s from the current cursor position and searches for the first occurrence of the delimiter `delim`, returning the text between the initial position and the delimiter found. If the delimiter is not found, it raises the exception `Not_found` without modifying the cursor, leaving the caller free to react to the failure without losing information about the current position.

```
let read_until delim =
  assert (String.length delim > 0);
  assert (!i >= 0 && !i <= len);
  let i_pre = !i in
  let dlen = String.length delim in
  let j = ref !i in
  while !j + dlen <= len && String.sub s !j dlen <> delim do
    incr j
  done;
  if !j + dlen > len then raise Not_found;
  let content = String.sub s !i (!j - !i) in
  i := !j + dlen;
  assert (!i > i_pre && !i <= len);
  content
```

Definition 5.1 (Hoare triples for `read_until`). Let $d = |\text{delim}|$ and $i_0 = !i$. The formal specification of `read_until` is expressed by two triples, one for each possible outcome.

When the delimiter is present starting from position i_0 :

$$\begin{aligned} & \{ 0 \leq i_0 \leq \ell \wedge d > 0 \wedge \exists j \geq i_0. j + d \leq \ell \wedge s[j \dots j+d) = \text{delim} \} \\ & \quad \text{read_until } \text{delim} \\ & \{ \text{result} = s[i_0 \dots j_{\min}) \wedge !i = j_{\min} + d \wedge !i > i_0 \wedge !i \leq \ell \} \end{aligned}$$

where $j_{\min}$ is the smallest $j \geq i_0$ such that $s[j \dots j+d) = \text{delim}$.

When the delimiter is absent:

$$\begin{aligned} & \{ 0 \leq i_0 \leq \ell \wedge d > 0 \wedge \forall j \geq i_0. j + d \leq \ell \Rightarrow s[j \dots j+d) \neq \text{delim} \} \\ & \quad \text{read_until } \text{delim} \\ & \{ \text{Not_found raised} \wedge !i = i_0 \} \end{aligned}$$

Remark 5.1. Among the two postconditions, the one for the failure case is the more subtle, because it is not immediately readable from the code. The cursor i remains unchanged at i_0 because the only instruction that modifies it, that is $i := !j + dlen$, appears after the check `if !j + dlen > len` which in the failure case causes an early exit via `raise Not_found`, making that line unreachable. This property is essential for the correctness of `try_delim`, which depends on the fact that a failed attempt of `read_until` leaves no trace on the state.

Proposition 5.1 (Partial correctness of `read_until`). *The function `read_until` satisfies the two Hoare triples stated above.*

Proof. The loop invariant of the `while` loop is the proposition

$$I(j) \equiv i_0 \leq j \leq \ell \wedge \forall k. i_0 \leq k < j \Rightarrow s[k \dots k+d) \neq \text{delim},$$

which holds vacuously at entry with $j = i_0$ and is preserved by the loop body, which increments j only when $s[j \dots j+d) \neq \text{delim}$.

Success case. The loop stops with $s[j \dots j+d) = \text{delim}$, and by $I(j)$ this j is the first such index $\geq i_0$, that is $j = j_{\min}$. The function returns $s[i_0 \dots j_{\min})$ and sets $!i = j_{\min} + d$. From $j_{\min} \geq i_0$ and $d \geq 1$ it follows that $!i > i_0$; from $j_{\min} + d \leq \ell$ it follows that $!i \leq \ell$.

Failure case. The loop stops with $j + d > \ell$, and by $I(j)$ no occurrence of `delim` exists in $s[i_0 \dots \ell-d]$. The function raises `Not_found` without having ever assigned to i , so $!i = i_0$. $\square$

5.3 The function `try_delim`

The function `try_delim` handles the entire sequence of operations needed to recognise a delimited construct, including recovery in the failure case. When the closing delimiter is not found, the text of the opening delimiter is reintegrated into the buffer as literal text so as not to lose input characters, and the condition $|fallback_str| = \text{open_len}$ present in the code as an `assert` guarantees that this reintegration is always correct.

```
let try_delim open_len delim make fallback_str =
  assert (String.length fallback_str = open_len);
  flush ();
  i := !i + open_len;
  (try acc := make (read_until delim) :: !acc
   with Not_found -> Buffer.add_string buf fallback_str)
```

Definition 5.2 (Hoare triples for `try_delim`). Let $i_0 = !i$, $ol = \text{open_len}$, $fb = \text{fallback_str}$ with $|fb| = ol$, and acc_0 the content of the accumulator before execution of `flush()`. The common precondition of the two triples is

$$P \equiv 0 \leq i_0 \leq \ell \wedge ol \geq 1 \wedge |fb| = ol \wedge i_0 + ol \leq \ell,$$

while the postconditions distinguish the two cases. When the closing delimiter is present starting from $i_0 + ol$:

$$\{P\} \text{ try_delim } \{ acc = \text{make}(\text{content}) :: acc_0 \wedge !i > i_0 \wedge buf = \varepsilon \}$$

When the closing delimiter is absent starting from $i_0 + ol$:

$$\{P\} \text{ try_delim } \{ buf = fb \wedge !i = i_0 + ol \wedge acc = acc_0 \}$$

Remark 5.2. The structure of the postcondition in the failure case. The instruction $i := !i + \text{open_len}$, which moves the cursor past the characters of the opening delimiter, is executed unconditionally before any attempt, so the cursor advances by ol in both outcomes. In the success case this advance is absorbed by the reading of the content and the closing delimiter, and the whole is represented in the accumulator by the node built by `make`. In the failure case the advance remains uncovered and is compensated by adding to the buffer the string fb of length ol , and this is the reason why the condition $|fb| = ol$, present in the code as an `assert`, is not a routine guard but the keystone of the global invariant that guarantees the preservation of every input character.

Proposition 5.2 (Partial correctness of `try_delim`). *The function `try_delim` satisfies the two triples stated above, assuming that `read_until` satisfies its own specification.*

Proof. Execution of `flush()` empties the buffer by transferring its contents to `acc`, after which the instruction $!i := i_0 + ol$ moves the cursor to the start of the putative delimited content and `read_until delim` is invoked.

Success case. By the specification of `read_until`, the call returns $\text{content} = s[i_0 + ol \dots j_{\min}]$ and moves the cursor to $j_{\min} + d$, which exceeds $i_0 + ol$ and hence i_0 ; the function `make` is applied to the content and the result is prepended to `acc`, while the buffer remains empty.

Failure case. By the specification of `read_until` in the not-found case, the cursor is not modified and remains at $i_0 + ol$; the exception is caught, fb is added to the buffer, and the accumulator remains acc_0 . $\square$

5.4 Global invariant of the lexer

The two Hoare triples compose into a global invariant that holds at the beginning and end of every iteration of the main loop of `Lexer.parse` and constitutes the most important correctness guarantee of the entire module, in the sense that every character of the input string contributes only once to the accumulated output.

Definition 5.3 (Global invariant of the lexer). We now denote by $\text{emitted}(\text{acc}, \text{buf})$ the string obtained by concatenating in the correct order the texts of all inlines in `acc` and the current content of the buffer `buf`; the global invariant is then the proposition

$$\text{Inv} \equiv 0 \leq !i \leq \ell \wedge s[0 \dots !i] = \text{emitted}(\text{acc}, \text{buf}),$$

which states that the prefix of s already consumed by the cursor coincides with what has been produced in the accumulator and the buffer.

Proposition 5.3. *Inv is an invariant of the main loop of `Lexer.parse`, in the sense that it is preserved by every branch of the loop.*

Proof. For constructs handled by `try_delim`, the postcondition guarantees that in the success case the cursor advances by $ol + |\text{content}| + d$ while an element whose text corresponds to those characters is added to the accumulator, and in the failure case the cursor advances by ol while ol characters are added to the buffer, so that in both situations $\text{emitted}(\text{acc}, \text{buf})$ grows as much as the cursor advances, preserving Inv .

For the two-phase link case, the most delicate branch is the one in which the opening parenthesis `(` is found but the closing `)` does not appear before the end of the string: in this case the postcondition of `read_until` ensures that the cursor stops immediately after `(`, and the reintegration through `String.sub s start (!i - start)` inserts into the

buffer the characters between $[]$ and the current position, compensating the advance and preserving Inv .

For an ordinary character, a single character is added to the buffer and the cursor increments by one, which makes the verification of Inv immediate. $\square$

6 Conclusions

We hope this work may be useful to someone; we refer to the bibliography for further reading.

Appendices

A Free Monoids and Formal Languages

The semantic domain **String** used throughout this document is the set of all finite strings over an alphabet Σ ; this structure needs to be clarified to understand why concatenation with neutral element ε is the right structure for modelling the text produced by a compiler.

Since a formal language over an alphabet A is a subset of A^* , and the theory of formal languages can be understood as the study of subsets of the free monoid over A , and the Kleene closure A^* is precisely the free monoid over A : every regular language over A is a subset of A^* closed under union, product, and submonoid generation [7]. In the document, $\mathbf{String} = \Sigma^*$ where Σ is the Unicode alphabet, the function $\mathcal{S}$ is a homomorphism of the free monoid $(\mathbf{String}, \cdot, \varepsilon)$ into itself, and the meaning functions $\mathcal{I}[\cdot]$, $\mathcal{B}[\cdot]$, $\mathcal{E}[\cdot]$ are all homomorphisms from the structured syntactic domain to this monoid, which guarantees that the semantics respects the algebraic structure of concatenation.

B Least Fixed Point and Kleene's Theorem

In the body of the document the Knaster-Tarski Theorem is used to guarantee the existence of the least fixed point of the functional Φ ; we note that it is useful to clarify the distinction between two results that are often cited together. The Knaster-Tarski Theorem states that every monotone function on a complete lattice has a least fixed point, and this result says nothing about how such a fixed point can be computed, nor does it require continuity of the function. Kleene's Theorem, or the fixed-point theorem for CPOs, states something more constructive: given D a CPO with least element $\perp$ and $f : D \rightarrow D$ a *continuous* function in the sense of Scott, that is monotone and preserving least upper bounds of directed chains, then the least fixed point of f is the limit of the chain

$$\perp \sqsubseteq f(\perp) \sqsubseteq f^2(\perp) \sqsubseteq \dots$$

that is $\text{fix}(f) = \bigsqcup_{n \geq 0} f^n(\perp)$ [8]. We note that this is the version applied in the document, since the continuity of Φ allows computing $\mathcal{E}[\cdot]$ as the limit of the approximations $\Phi^n(\perp_0)$, each of which is defined on lists of increasing length, and continuity is the condition that transforms an existence result into a computation by successive approximations.

References

- [1] R. Barbuti, P. Mancarella, F. Turini. *Elementi di Semantica Operazionale*. Lecture notes for Fondamenti di Programmazione, B.Sc. in Computer Science, University of Pisa, A.A. 2004/05. <https://pages.di.unipi.it/mancarella/FP/materiale/nuovasemop.pdf>
- [2] G. D. Plotkin. *A structural approach to operational semantics*. Technical Report DAIMI FN-19, Aarhus University, 1981. Reprinted in *J. Log. Algebr. Program.*, 60–61:17–139, 2004.
- [3] G. Winskel. *The Formal Semantics of Programming Languages: An Introduction*. MIT Press, Cambridge, MA, 1993. ISBN 0-262-73103-7. <https://direct.mit.edu/books/monograph/4338/The-Formal-Semantics-of-Programming-LanguagesAn>

- [4] P. Mancarella. *Sintassi dei Linguaggi di Programmazione*. Dispensa per il corso di Fondamenti di Programmazione, Università di Pisa. <https://pages.di.unipi.it/mancarella/FP/materiale/syntax.pdf>
- [5] C. A. R. Hoare. An axiomatic basis for computer programming. *Communications of the ACM*, 12(10):576–580, 1969.
- [6] R. W. Floyd. Assigning meanings to programs. *Proceedings of a Symposium on Applied Mathematics*, 19:19–32, 1967.
- [7] Wikipedia contributors. *Free monoid*. Wikipedia, The Free Encyclopedia. https://en.wikipedia.org/wiki/Free_monoid
- [8] Wikipedia contributors. *Kleene fixed-point theorem*. Wikipedia, The Free Encyclopedia. https://en.wikipedia.org/wiki/Kleene_fixed-point_theorem